

INTERFACE

Configure your terminal for Claude Code

Fix Shift+Enter for newlines, get a terminal bell when Claude finishes, configure tmux, match the color theme, and enable Vim mode in the Claude Code CLI.

Claude Code works in any terminal without configuration. This page is for when something specific is not behaving the way you expect. Find your symptom below. If everything already feels right, you do not need this page.

- [Shift+Enter submits instead of inserting a newline](#)
- [Option-key shortcuts do nothing on macOS](#)
- [No sound or alert when Claude finishes](#)
- [You run Claude Code inside tmux](#)
- [Display flickers or scrollbar jumps](#)
- [You want Vim keys in the prompt](#)

This page is about getting your terminal to send the right signals to Claude Code. To change which keys Claude Code itself responds to, see [keybindings](#) instead.

Enter multiline prompts

Pressing Enter submits your message. To add a line break without submitting, press Ctrl+J, or type `\` and then press Enter. Both work in every terminal with no setup.

In most terminals you can also press Shift+Enter, but support varies by terminal emulator:

Terminal	Shift+Enter for newline
Ghostty, Kitty, iTerm2, WezTerm, Warp, Apple Terminal, Windows Terminal	Works without setup
VS Code, Cursor, Devin Desktop, Alacritty, Zed	Run <code>/terminal-setup</code> once
gnome-terminal, JetBrains IDEs such as PyCharm and Android Studio	Not available; use Ctrl+J or <code>\</code> then Enter

For VS Code, Cursor, Devin Desktop, Alacritty, and Zed, `/terminal-setup` writes Shift+Enter and other keybindings into the terminal’s configuration file. Existing bindings are left in place; if you see a message such as `VSCode terminal Shift+Enter key binding already configured`, no change was made. Run `/terminal-setup` directly in the host terminal rather than inside tmux or screen, since it needs to write to the host terminal’s configuration.

In VS Code, Cursor, and Devin Desktop, `/terminal-setup` also updates two editor settings: it sets `terminal.integrated.gpuAcceleration` to `"off"` to prevent garbled text in the integrated terminal, and it sets `terminal.integrated.mouseWheelScrollSensitivity` for smoother scrolling in fullscreen mode. To undo the GPU acceleration change, set it back to `"auto"` and reload the editor window.

If you are running inside tmux, Shift+Enter also requires the tmux configuration below even when the outer terminal supports it.

To bind newline to a different key, or to swap behavior so Enter inserts a newline and Shift+Enter submits, map the `chat:newline` and `chat:submit` actions in your keybindings file.

Enable Option key shortcuts on macOS

Some Claude Code shortcuts use the Option key, such as Option+Enter for a newline or Option+P to switch models. On macOS, most terminals do not send Option as a modifier by default, so these shortcuts do nothing until you enable it. The terminal setting for this is usually labeled “Use Option as Meta Key”; Meta is the historical Unix name for the key now labeled Option or Alt.

Apple Terminal

Open Settings → Profiles → Keyboard and check “Use Option as Meta Key”.If you accepted Claude Code’s first-run prompt that offered “Option+Enter for newlines and visual bell”, this is already done. That prompt runs `/terminal-setup` for you, which enables Option as Meta and switches the audio

bell to a visual screen flash in your Apple Terminal profile.

iTerm2

Open Settings → Profiles → Keys → General and set Left Option key and Right Option key to “Esc+”. Running `/terminal-setup` in iTerm2 enables “Applications in terminal may access clipboard” under Settings → General → Selection so the `/copy` command can write to your system clipboard. The command detects iTerm2 even when run from inside tmux. Restart iTerm2 for the change to take effect.

VS Code

Add `"terminal.integrated.macOptionIsMeta": true` to your VS Code settings.

```
CLAUDE_CODE_NO_FLICKER=1 claude
```

For Ghostty, Kitty, and other terminals, look for an Option-as-Alt or Option-as-Meta setting in the terminal's configuration file.

Get a terminal bell or notification

When Claude finishes a task or pauses for a permission prompt, it fires a notification event. Surfacing this as a terminal bell or desktop notification lets you switch to other work while a long task runs.

By default Claude Code sends a desktop notification only in Ghostty, Kitty, and iTerm2. In other terminals, set `preferredNotifChannel` to `"terminal_bell"` to ring the terminal bell instead, or configure a **Notification hook** for a custom sound or command.

The desktop notification reaches your local machine over SSH, so a remote session can still alert you. Ghostty and Kitty forward it to your OS notification center without further setup. iTerm2 requires you to enable forwarding:

1 Open iTerm2 notification settings

Go to Settings → Profiles → Terminal.

2 Enable alerts

Check “Notification Center Alerts”, then click “Filter Alerts” and enable “Send escape sequence-generated alerts”.

If notifications still do not appear, confirm that your terminal application has notification permission in your OS settings, and if you are running inside tmux, **enable passthrough**.

Play a sound with a Notification hook

In any terminal you can configure a **Notification hook** to play a sound or run a custom command when Claude needs your attention. Hooks run alongside the built-in notification rather than replacing it, so terminals that do not receive a desktop notification, such as Warp or the VS Code integrated terminal, can use a hook or set `preferredNotifChannel` to `"terminal_bell"` instead.

The example below plays a system sound on macOS. The linked guide has desktop notification commands for macOS, Linux, and Windows.

~/claude/settings.json

```
{
  "hooks": {
    "Notification": [
      {
        "hooks": [{ "type": "command", "command": "afplay
/System/Library/Sounds/Glass.aiff" }]
      }
    ]
  }
}
```

Configure tmux

When Claude Code runs inside tmux, two things break by default: Shift+Enter submits instead of inserting a newline, and desktop notifications and the **progress bar** never reach the outer terminal. Add these lines to `~/tmux.conf`, then run `tmux source-file ~/tmux.conf` to apply them to the running server:

~/tmux.conf

```
set -g allow-passthrough on
set -s extended-keys on
set -as terminal-features 'xterm*:extkeys'
```

The `allow-passthrough` line lets notifications and progress updates reach the outer terminal instead of being swallowed by tmux. The `extended-keys` lines let tmux distinguish Shift+Enter from plain Enter so the newline shortcut works.

Match the color theme

Use the `/theme` command, or the theme picker in `/config`, to choose a Claude Code theme that matches your terminal. Selecting the auto option detects your terminal's light or dark background, so the theme follows OS appearance changes whenever your terminal does. Claude Code does not control the terminal's own color scheme, which is set by the terminal application.

To customize what appears at the bottom of the interface, configure a **custom status line** that shows the current model, working directory, git branch, or other context.

Create a custom theme

⚠ Custom themes require Claude Code v2.1.118 or later.

In addition to the built-in presets, `/theme` lists any custom themes you have defined and any themes contributed by installed **plugins**. Select **New custom theme...** at the end of the list to create one interactively: you name the theme, then pick individual color tokens to override. Press `Ctrl+E` while a custom theme is highlighted to edit it.

Each custom theme is a JSON file in `~/.claude/themes/`. The filename without the `.json` extension is the theme's slug, and selecting the theme stores `custom:<slug>` as your theme preference. The file has three optional fields:

Field	Type	Description
<code>name</code>	string	Display label shown in <code>/theme</code> . Defaults to the filename slug
<code>base</code>	string	Built-in preset the theme starts from: <code>dark</code> , <code>light</code> , <code>dark-daltonized</code> , <code>light-daltonized</code> , <code>dark-ansi</code> , or <code>light-ansi</code> . Defaults to <code>dark</code>
<code>overrides</code>	object	Map of color token names to color values. Tokens not listed here fall through to the base preset

Color values accept `#rrggbb`, `#rgb`, `rgb(r,g,b)`, `ansi256(n)`, or `ansi:<name>` where `<name>` is one of the 16 standard ANSI color names such as `red` or `cyanBright`. Unknown tokens and invalid color values are ignored, so a typo cannot break rendering.

The following example defines a theme that keeps the dark preset but recolors the prompt accent, error text, and success text:

```
~/.claude/themes/dracula.json

{
  "name": "Dracula",
  "base": "dark",
  "overrides": {
    "claude": "#bd93f9",
    "error": "#ff5555",
    "success": "#50fa7b"
  }
}
```

Claude Code watches `~/ .claude/themes/` and reloads when a file changes, so edits made in your editor apply to a running session without a restart.

The reference below covers the tokens you can set in `overrides`. The interactive editor in `/theme` shows the same tokens with a live preview, plus a few single-purpose accents such as onboarding screen colors that are omitted here.

► Color token reference

Switch to fullscreen rendering

If the display flickers or the scroll position jumps while Claude is working, switch to **fullscreen rendering mode**. It draws to a separate screen the terminal reserves for full-screen apps instead of appending to your normal scrollbar, which keeps memory usage flat and adds mouse support for scrolling and selection. In this mode you scroll with the mouse or PageUp inside Claude Code rather than with your terminal's native scrollbar; see the **fullscreen page** for how to search and copy.

Run `/tui fullscreen` to switch in the current session with your conversation intact. To make it the default, set the `CLAUDE_CODE_NO_FLICKER` environment variable before starting Claude Code:

Paste large content

When you paste more than 10,000 characters into the prompt, Claude Code collapses the input to a `[Pasted text]` placeholder so the input box stays usable. The full content is still sent to Claude when you submit.

The VS Code integrated terminal can drop characters from very large pastes before they reach Claude Code, so prefer file-based workflows there. For very large inputs such as entire files or long logs, write the content to a file and ask Claude to read it instead of pasting. This keeps the conversation transcript readable and lets Claude reference the file by path in later turns.

Edit prompts with Vim keybindings

Claude Code includes a Vim-style editing mode for the prompt input. Enable it through `/config` → Editor mode, or by setting `editorMode` to `"vim"` in `~/ .claude/settings.json`. Set Editor mode back to `normal` to turn it off.

Vim mode supports a subset of NORMAL- and VISUAL-mode motions and operators, such as `hjk1` navigation, `v / V` selection, and `d / c / y` with text objects. See the [Vim editor mode reference](#) for the full key table. Vim motions are not remappable through the keybindings file.

Pressing Enter still submits your prompt in INSERT mode, unlike standard Vim. Use `o` or `O` in NORMAL mode, or Ctrl+J, to insert a newline instead.

Related resources

- [Interactive mode](#): full keyboard shortcut reference and the Vim key table
- [Keybindings](#): remap any Claude Code shortcut, including Enter and Shift+Enter
- [Fullscreen rendering](#): details on scrolling, search, and copy in fullscreen mode
- [Hooks guide](#): more Notification hook examples for Linux and Windows
- [Troubleshooting](#): fixes for issues outside terminal configuration